CI/CD, Jenkins, Maven & Docker Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Test Automation Engineer with 4+ years of experience

Questions in your original document	**123**
Exact duplicates removed (same question, different wording of punctuation or case)	**1**
Near-duplicates merged (same question asked in different words)	**60**
Unique questions in this PDF	**62**

Section 1: Critical (Must Prepare)	18 questions
Section 2: High Priority	24 questions
Section 3: Medium Priority	16 questions
Section 4: Low Priority / Quick Revision	4 questions

How to Use This Document

Questions are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Prepare) 18 questions	Asked in almost every automation interview at 4+ years. Describe how it works in YOUR project: which tool, which pipeline stages, what triggers runs and how failures are handled.
High Priority 24 questions	Frequent practical questions. Answer with the setup steps and one real example from your Jenkins/Maven/Docker work.
Medium Priority 16 questions	Occasionally asked, usually as follow-ups or when your resume mentions the tool.
Low Priority / Quick Revision 4 questions	Basic commands and rarely asked details. Skim them so you are never caught off guard.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
CI/CD Concepts & QA's Role	Can you explain the pipeline stages and exactly where your automation runs (smoke, regression, quality gates)?
Jenkins	Hands-on: freestyle vs pipeline, Jenkinsfile, parameters, scheduling with cron, triggering from Git, parallel jobs.
Reports, Failures & Troubleshooting	How tests are reported and what you do when a build fails or tests pass locally but fail in CI.
Maven & Build Management	Lifecycle, pom.xml, Surefire, dependencies and environment profiles.
Docker for QA	Running browsers and test environments in containers, Dockerfile and Compose basics.
Linux / CLI	Basic commands to inspect logs, processes and disk on a test server.

Note: short fragments in your file (for example 'Handling test failures' or 'Scheduling builds (cron)') are phrased as full questions and merged with matching ones. Docker and Linux commands are ranked lower because they are usually asked only as follow-ups.

Table of Contents

Section 1: Critical (Must Prepare) (18 questions) . . . 5
CI/CD Concepts & QA's Role (6) . . . 5
Jenkins: Jobs, Pipelines, Triggers & Scheduling (6) . . . 6
Reports, Failures & Troubleshooting in CI (4) . . . 7
Maven & Build Management (2) . . . 8

Section 2: High Priority (24 questions) . . . 9
CI/CD Concepts & QA's Role (5) . . . 9
Jenkins: Jobs, Pipelines, Triggers & Scheduling (5) . . . 9
Reports, Failures & Troubleshooting in CI (2) . . . 10
Maven & Build Management (3) . . . 11
Docker for QA Automation (6) . . . 11
Linux / CLI Basics for QA (3) . . . 12

Section 3: Medium Priority (16 questions) . . . 14
CI/CD Concepts & QA's Role (4) . . . 14
Jenkins: Jobs, Pipelines, Triggers & Scheduling (2) . . . 14
Docker for QA Automation (8) . . . 15
Linux / CLI Basics for QA (2) . . . 16

Section 4: Low Priority / Quick Revision (4 questions) . . . 17
Jenkins: Jobs, Pipelines, Triggers & Scheduling (1) . . . 17
Docker for QA Automation (1) . . . 17
Linux / CLI Basics for QA (2) . . . 17

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	S4	Total
CI/CD Concepts & QA's Role	6 Qs, p.5	5 Qs, p.9	4 Qs, p.14	-	**15**
Jenkins: Jobs, Pipelines, Triggers & Scheduling	6 Qs, p.6	5 Qs, p.9	2 Qs, p.14	1 Qs, p.17	**14**
Reports, Failures & Troubleshooting in CI	4 Qs, p.7	2 Qs, p.10	-	-	**6**
Maven & Build Management	2 Qs, p.8	3 Qs, p.11	-	-	**5**
Docker for QA Automation	-	6 Qs, p.11	8 Qs, p.15	1 Qs, p.17	**15**
Linux / CLI Basics for QA	-	3 Qs, p.12	2 Qs, p.16	2 Qs, p.17	**7**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Prepare) (18 questions)

Asked in almost every automation interview at 4+ years. Describe how it works in YOUR project: which tool, which pipeline stages, what triggers runs and how failures are handled.

CI/CD Concepts & QA's Role (6)

1. What is CI/CD and why is it important in automation and for QA engineers? **[asked x3 in your bank]**

   > **Answer:** CI (Continuous Integration) means code is merged often and every change triggers an automated build and tests. CD delivers the tested build to environments automatically. For QA it gives fast feedback, catches defects early, runs regression on every build and removes manual, inconsistent test runs.
   >
   > **Example:** Every commit to the checkout service triggers the smoke suite within minutes.

2. What CI/CD tools have you used? What tools can be used for CI/CD in QA automation? **[asked x2 in your bank]**

   > **Answer:** Common CI/CD tools: Jenkins, GitHub Actions, GitLab CI, Azure DevOps, CircleCI, TeamCity and Bamboo. Name the ones you really used (for example Jenkins with Maven and Git) and what you did in them.
   >
   > **Example:** Jenkins pipelines running Selenium and API suites for the shop, with Docker for browsers.

3. Explain the CI/CD pipeline and its common stages from a QA perspective. **[asked x2 in your bank]**

   > **Answer:** Typical stages: commit, build and unit tests, static analysis, deploy to the test environment, smoke tests, API/UI regression (often in parallel), reports and quality gate, deploy to staging and then production with monitoring. QA owns the test and quality-gate stages.
   >
   > **Example:** Smoke on every build, full regression nightly, results posted to the team.

   ```
   pipeline {
     agent any
     stages {
       stage('Build')      { steps { sh 'mvn -B clean compile' } }
       stage('Smoke')      { steps { sh 'mvn test -Dgroups=smoke' } }
       stage('Regression') { steps { sh 'mvn test -Dgroups=regression' } }
     }
   }
   ```

4. What is the role of QA in a CI/CD pipeline, and how does CI/CD improve software quality? **[asked x2 in your bank]**

   > **Answer:** QA defines and automates the tests that run in the pipeline, sets quality gates, keeps tests stable and fast, triages failures and reports quality metrics. CI/CD improves quality through early feedback, consistent regression, repeatable environments and quick rollback signals.
   >
   > **Example:** A failing payment smoke test stops the build before it reaches QA.

5. How do you integrate automated tests, scripts and your framework into a CI/CD pipeline? Is your framework CI/CD ready? **[asked x5 in your bank]**

   > **Answer:** A CI-ready framework runs from the command line (mvn test), headless, with configuration through system properties, independent and parallel-safe tests, JUnit/TestNG XML results, reports and no local paths. Steps: Git repo, Jenkins job, build command, report publishing, notifications.
   >
   > **Example:** mvn clean test -Denv=qa -Dbrowser=chrome -Dheadless=true runs the same on a laptop and Jenkins.

   ```
   mvn -B clean test -Denv=qa -Dbrowser=chrome -Dheadless=true -DsuiteXmlFile=smoke.xml
   ```

6. How do you integrate your Selenium suite with Jenkins, GitHub Actions or Azure DevOps? **[asked x3 in your bank]**

> **Answer:** Run the Maven test command on a CI agent with headless Chrome (or a Grid/Docker browser), then publish reports and screenshots. The steps are the same in Jenkins, GitHub Actions and Azure DevOps; only the pipeline syntax changes.
>
> **Example:** GitHub Actions workflow runs the UI smoke suite on every pull request.

```
name: ui-tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 17 }
      - run: mvn -B clean test -Dheadless=true
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: reports, path: target/ }
```

Jenkins: Jobs, Pipelines, Triggers & Scheduling (6)

7. What is Jenkins and why is it important for QA? How and when do you use it in your automation? **[asked x4 in your bank]**

> **Answer:** Jenkins is an open-source automation server for CI/CD. QA uses it to run suites on triggers or schedules, run on remote agents, publish reports and notify the team.
>
> **Example:** Jenkins runs the nightly regression for the shop and emails the report.

8. What is the difference between a Freestyle job and a Pipeline job? Explain the basics of a Jenkins pipeline. **[asked x4 in your bank]**

> **Answer:** Freestyle jobs are configured through the UI, are simple but limited and not version controlled. Pipeline jobs define the flow as code (Jenkinsfile) with stages, parallel steps, conditions and post actions, and are stored in Git. Declarative pipelines use agent, stages, steps and post blocks.
>
> **Example:** Freestyle for a one-step API run; Pipeline for build, smoke, regression and report stages.

```
pipeline {
  agent any
  stages {
    stage('Test') { steps { sh 'mvn clean test' } }
  }
  post { always { junit 'target/surefire-reports/*.xml' } }
}
```

9. What is a Jenkinsfile and why does QA use it? **[asked x3 in your bank]**

> **Answer:** A Jenkinsfile is a text file in the repository that defines the pipeline as code. QA uses it so the test pipeline is versioned, reviewed through pull requests and easy to reuse.
>
> **Example:** The UI regression pipeline lives next to the test code and changes with it.

```
// Jenkinsfile (checked in at the repo root)
pipeline { agent any
  stages { stage('Regression') { steps { sh 'mvn clean test -Dgroups=regression' } } }
}
```

10. How do you create and configure a Jenkins job to run (trigger) your automation suite? **[asked x5 in your bank]**

> **Answer:** Create a New Item (Pipeline or Freestyle), set the Git repo, credentials and branch, add a trigger (cron, poll SCM or webhook), add the Maven build step with the suite, then add post-build actions to publish reports and notify.
>
> **Example:** A freestyle job runs the testng.xml for the checkout module every night.

```
# Freestyle build step (Invoke top-level Maven targets / Execute shell)
mvn clean test -DsuiteXmlFile=testng.xml -Denv=qa
```

11. How do you parameterize a Jenkins build / pass parameters to a job? **[asked x2 in your bank]**

> **Answer:** Enable "This project is parameterized" (or a parameters block) and add choice, string or boolean parameters; read them as params.NAME in a pipeline or ${NAME} in a shell step.
>
> **Example:** ENV (qa/uat) and BROWSER parameters decide where and how the suite runs.

```
parameters {
  choice(name: 'ENV', choices: ['qa', 'uat'], description: 'Target environment')
  string(name: 'BROWSER', defaultValue: 'chrome')
  booleanParam(name: 'HEADLESS', defaultValue: true)
}
// sh "mvn clean test -Denv=${params.ENV} -Dbrowser=${params.BROWSER}"
```

12. How do you schedule a nightly regression run in Jenkins (cron)? **[asked x2 in your bank]**

> **Answer:** Use a cron trigger (Build periodically or triggers { cron(...) }). Jenkins' H symbol spreads load. Format: minute hour day-of-month month day-of-week.
>
> **Example:** Run regression at about 2 AM Monday to Friday.

```
triggers { cron('H 2 * * 1-5') }
```

Reports, Failures & Troubleshooting in CI (4)

13. How are automation reports published and stored in Jenkins? How do you publish HTML and TestNG reports and share them? **[asked x5 in your bank]**

> **Answer:** Use plugins: HTML Publisher for HTML reports, the TestNG/JUnit result plugins for test results, and Allure or Extent reports. Reports are kept with the build, archived and shared by link or email.
>
> **Example:** Extent HTML report published for each regression build.

```
publishHTML(target: [reportDir: 'target', reportFiles: 'ExtentReport.html',
  reportName: 'Extent Report', keepAll: true])
junit 'target/surefire-reports/*.xml'
archiveArtifacts artifacts: 'screenshots/**', allowEmptyArchive: true
```

14. How do you handle failed builds and test failures in Jenkins? **[asked x3 in your bank]**

> **Answer:** Open the console log and test report, classify the failure (build/compile, environment, real defect, flaky test), check screenshots and logs, rerun, then fix the script or raise a defect and notify the owner.
>
> **Example:** Build failed due to an unreachable QA URL: reported as an environment issue and rerun after recovery.

15. Tests pass locally but fail on Jenkins/CI nodes. How do you debug? **[asked x2 in your bank]**

> **Answer:** Common causes: headless/viewport differences, slower machine (timing), different browser/driver versions, data or environment differences, OS paths and parallel runs. Debug with logs and screenshots, run in the same Docker image locally, align versions and set explicit waits and window size.
>
> **Example:** Hover menu works locally but the headless CI window is small: set --window-size=1920,1080.

16. How do you handle flaky or unstable tests in CI/CD pipelines? **[asked x2 in your bank]**

> **Answer:** Find the root cause (waits, shared data, environment), fix it, isolate test data, rerun failures a small number of times, quarantine unstable tests and track the flaky rate.
>
> **Example:** Surefire reruns a failed payment test twice before marking it failed.

```
mvn test -Dsurefire.rerunFailingTestsCount=2
```

Maven & Build Management (2)

17. What is Maven and why do you use it in automation projects? What is pom.xml and which basic Maven commands do you use? **[asked x3 in your bank]**

> **Answer:** Maven is a build and dependency management tool. pom.xml defines project details, dependencies, plugins and profiles. Common commands: mvn clean, compile, test, package, install and mvn test -Dtest=LoginTest.
>
> **Example:** pom.xml holds Selenium, TestNG and Rest Assured dependencies for the shop framework.

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.25.0</version>
</dependency>
<!-- mvn clean test | mvn clean install -DskipTests | mvn test -Dtest=LoginTest -->
```

18. What is the Maven build lifecycle? **[asked x2 in your bank]**

> **Answer:** The default lifecycle phases are validate, compile, test, package, verify, install and deploy; running a phase runs all earlier phases. There is also the clean lifecycle (clean) and site. For automation you mostly run mvn test.
>
> **Example:** mvn clean test compiles the project, then runs the tests.

Section 2: High Priority (24 questions)

Frequent practical questions. Answer with the setup steps and one real example from your Jenkins/Maven/Docker work.

CI/CD Concepts & QA's Role (5)

19. What is Continuous Integration (CI) and what is Continuous Delivery (CD)? **[asked x2 in your bank]**

> **Answer:** Continuous Integration: developers merge frequently and each change is built and tested automatically. Continuous Delivery: every successful build is kept ready to release, usually with a manual approval before production.
>
> **Example:** CI runs unit and smoke tests on each commit; CD prepares a release candidate for staging.

20. What is the difference between Continuous Delivery and Continuous Deployment?

> **Answer:** Continuous Delivery keeps releases ready but a person approves the production deployment. Continuous Deployment deploys to production automatically whenever the pipeline passes, with no manual step.
>
> **Example:** Delivery: release manager approves the sale-day release. Deployment: every green commit goes live.

21. How do you trigger a CI/CD pipeline automatically after code changes (Git triggers)? **[asked x3 in your bank]**

> **Answer:** Use a webhook from GitHub/GitLab to Jenkins (or Poll SCM), a multibranch pipeline for branches and pull requests, or on: push in GitHub Actions.
>
> **Example:** A push to the develop branch starts the build and smoke tests.

```
triggers { pollSCM('H/5 * * * *') }   // or a GitHub webhook (preferred)
```

22. What are quality gates in CI/CD and how does QA use them?

> **Answer:** A quality gate is a pass/fail rule the pipeline must meet to continue, for example all smoke tests pass, regression pass rate above a threshold, no critical defects, coverage and static-analysis limits. QA sets thresholds and fails the build when they are not met.
>
> **Example:** Block deployment to staging if any payment test fails.

```
timeout(time: 10, unit: 'MINUTES') {
  def gate = waitForQualityGate()   // SonarQube plugin
  if (gate.status != 'OK') error "Quality gate failed: ${gate.status}"
}
```

23. How do you manage test data in CI/CD?

> **Answer:** Create data through API/DB scripts before the run, make it unique per build, keep environment-specific data files, seed the DB in a container, clean up afterwards and keep secrets in Jenkins credentials.
>
> **Example:** A setup stage creates test customers by API and a cleanup stage deletes them.

Jenkins: Jobs, Pipelines, Triggers & Scheduling (5)

24. How do you run tests in parallel in Jenkins (parallel jobs)? **[asked x3 in your bank]**

> **Answer:** Use parallel stages in the pipeline, split the suite into groups/jobs, or run TestNG/Grid/Docker in parallel; make sure tests are independent.
>
> **Example:** Cart, Payment and Search suites run at the same time.

```
stage('Tests') {
  parallel {
    stage('Cart')    { steps { sh 'mvn test -Dgroups=cart' } }
    stage('Payment') { steps { sh 'mvn test -Dgroups=payment' } }
  }
}
```

25. How do you run a Maven project in Jenkins?

> **Answer:** Configure JDK and Maven in Global Tool Configuration, point the job to the Git repo, and run goals such as clean test with the Maven build step or sh 'mvn clean test'.
>
> **Example:** Pipeline uses Maven 3 and JDK 17 to run the checkout suite.

```
pipeline {
  agent any
  tools { maven 'Maven3'; jdk 'JDK17' }
  stages { stage('Test') { steps { sh 'mvn -B clean test' } } }
}
```

26. How do you integrate Postman API tests in Jenkins?

> **Answer:** Install Node and Newman on the agent, run the collection with the environment file, produce JUnit/HTML reports and publish the JUnit results.
>
> **Example:** API smoke collection runs after every deployment.

```
sh 'newman run shop.postman_collection.json -e qa.postman_environment.json -r cli,junit'
junit 'newman/*.xml'   // publish the results
```

27. How do you use Jenkins to test across multiple environments?

> **Answer:** Add an ENV parameter, keep a property file or Maven profile per environment and use separate credentials, then run the same pipeline for QA, UAT and staging (or one stage per environment).
>
> **Example:** Same regression suite runs on QA first and on UAT after approval.

```
choice(name: 'ENV', choices: ['qa', 'uat', 'staging'])
// sh "mvn clean test -Denv=${params.ENV}"
```

28. Which Jenkins plugins have you used in test automation?

> **Answer:** Pipeline, Git, Maven Integration, HTML Publisher, TestNG Results, JUnit, Allure, Extent Reports, Email Extension, Slack Notification, Docker Pipeline, Credentials Binding, Parameterized Trigger and SonarQube Scanner.
>
> **Example:** Allure plugin shows trends and failure categories for the regression runs.

Reports, Failures & Troubleshooting in CI (2)

29. How do you archive test artifacts in Jenkins? **[asked x2 in your bank]**

> **Answer:** Use archiveArtifacts to store reports, screenshots and logs with the build, and a build discarder to limit retention.
>
> **Example:** Failed-test screenshots are kept for 30 builds.

```
archiveArtifacts artifacts: 'target/**/*.html, screenshots/**', fingerprint: true
options { buildDiscarder(logRotator(numToKeepStr: '30')) }
```

30. How do you set up email and Slack notifications after builds? **[asked x2 in your bank]**

> **Answer:** Use the Email Extension plugin (emailext) and the Slack plugin (slackSend) in the post section; configure SMTP and a Slack token in Jenkins credentials.
>
> **Example:** A failed nightly build posts the report link to the QA Slack channel.

```
post {
  failure {
    emailext subject: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
             body: "Report: ${env.BUILD_URL}", to: 'qa-team@shop.com'
    slackSend channel: '#qa-alerts', message: "Build failed: ${env.BUILD_URL}"
  }
}
```

Maven & Build Management (3)

31. What is the Maven Surefire plugin and where and why do you use it?

> **Answer:** Surefire is the Maven plugin that runs tests in the test phase. Configure it to run a testng.xml, set parallel options, pass system properties and rerun failed tests.
>
> **Example:** Surefire runs smoke.xml and passes the environment name to the tests.

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <suiteXmlFiles><suiteXmlFile>testng.xml</suiteXmlFile></suiteXmlFiles>
    <rerunFailingTestsCount>1</rerunFailingTestsCount>
  </configuration>
</plugin>
```

32. How do you manage different environments (QA, pre-production, production) with different data using Maven / pom.xml? **[asked x2 in your bank]**

> **Answer:** Define Maven profiles (or per-environment property files) for QA, pre-production and production and select one with -P or -Denv; keep environment data in separate files and never run destructive tests in production.
>
> **Example:** mvn test -Pqa uses the QA URL and QA test data.

```
<profiles>
  <profile>
    <id>qa</id>
    <properties><env>qa</env><base.url>https://qa.shop.com</base.url></properties>
  </profile>
</profiles>
<!-- mvn clean test -Pqa -->
```

33. How are dependencies handled in Maven at run time? If someone deleted all dependencies from the pom file, can you still execute tests? **[asked x2 in your bank]**

> **Answer:** Maven downloads dependencies from a remote repository (Maven Central) into the local ~/.m2 repository on first use and builds the classpath from the pom. If all dependencies were deleted from the pom, the test code would not compile (missing imports), so Maven could not run the tests.
>
> **Example:** Removing selenium-java from the pom breaks compilation of every page object.

Docker for QA Automation (6)

34. What is Docker and why is it useful in QA testing? How is it different from virtual machines? **[asked x2 in your bank]**

> **Answer:** Docker packages an application with its dependencies into containers. For QA it gives consistent environments, quick setup of browsers, databases and apps, isolation and parallel runs. Unlike VMs, containers share the host OS kernel, so they are lighter and start in seconds.
>
> **Example:** Start Chrome and a MySQL database in containers instead of installing them on every machine.

35. What is the difference between a Docker image and a container?

> **Answer:** An image is a read-only template built from a Dockerfile (layers). A container is a running instance of an image.
>
> **Example:** Build the test-runner image once, then start many containers from it.

```
docker build -t shop-tests .
docker run --rm shop-tests
```

36. How do you write a Dockerfile and run a Docker container for automation testing (including API testing)? **[asked x3 in your bank]**

> **Answer:** Write a Dockerfile with a base image (Maven + JDK), copy the pom and source, and run the tests as the command; build the image and run the container with environment variables and a volume for reports. API tests work the same way, only without a browser.
>
> **Example:** Run the API regression in a container on any machine or CI agent.

```
FROM maven:3.9-eclipse-temurin-17
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline
COPY src ./src
CMD ["mvn", "-B", "clean", "test"]
# docker build -t shop-tests .
# docker run --rm -e ENV=qa -v $(pwd)/target:/app/target shop-tests
```

37. How do you run Selenium in Docker and use Docker for cross-browser testing? **[asked x2 in your bank]**

> **Answer:** Run the official Selenium images (standalone-chrome or a hub with browser nodes) and connect the tests with RemoteWebDriver to http://localhost:4444; add Chrome, Firefox and Edge nodes for cross-browser runs.
>
> **Example:** Chrome and Firefox nodes run the checkout tests in parallel.

```
docker run -d -p 4444:4444 -p 7900:7900 --shm-size="2g" selenium/standalone-chrome
# test: new RemoteWebDriver(new URL("http://localhost:4444"), new ChromeOptions());
```

38. How do you integrate Docker with Jenkins for QA automation?

> **Answer:** Use a Jenkins agent with Docker and the Docker Pipeline plugin: run stages inside a container (agent { docker { image ... } }) or start Selenium Grid/DB containers with docker compose in the pipeline.
>
> **Example:** The test stage runs inside a Maven container, so every build uses the same tools.

```
pipeline {
  agent { docker { image 'maven:3.9-eclipse-temurin-17' } }
  stages { stage('Test') { steps { sh 'mvn -B clean test' } } }
}
```

39. How do you share test environments using Docker Compose?

> **Answer:** Docker Compose describes a multi-container environment (app, database, Selenium) in docker-compose.yml. Start it with docker compose up -d and stop it with down; keeping the file in Git gives everyone the same environment.
>
> **Example:** One compose file starts the shop app, its MySQL database and a Chrome node.

```
services:
  app:
    image: shop-app:latest
    ports: ["8080:8080"]
    depends_on: [db]
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: pass
      MYSQL_DATABASE: shop
  chrome:
    image: selenium/standalone-chrome
    ports: ["4444:4444"]
```

Linux / CLI Basics for QA (3)

40. What is the CLI and why should QA engineers know it?

> **Answer:** The CLI (command-line interface) lets you run commands in a terminal. QA uses it to run tests (mvn), inspect logs, manage servers and Docker, use Git and work on CI agents, and it is faster and scriptable.
>
> **Example:** Check the app log on the QA server over SSH to find why an order failed.

41. How do you search inside a file for a keyword (grep)?

> **Answer:** Use grep with the keyword and file; useful options: -i ignore case, -n line numbers, -r recursive, -c count, -A/-B lines of context.
>
> **Example:** Find ERROR lines for a failed order in the application log.

```
grep "ERROR" app.log
grep -in "order 1001" app.log
grep -rn "Timeout" /var/log/shop/ -A 2
```

42. How do you monitor logs in real time (tail -f)?

> **Answer:** Use tail -f to follow a log as it grows; tail -n shows the last lines; pipe to grep to filter.
>
> **Example:** Watch the log while placing an order in the UI.

```
tail -f app.log
tail -n 100 app.log
tail -f app.log | grep ERROR
```

Section 3: Medium Priority (16 questions)

Occasionally asked, usually as follow-ups or when your resume mentions the tool.

CI/CD Concepts & QA's Role (4)

43. What is a blue-green deployment and how does it help QA?

> **Answer:** Blue-green keeps two identical production environments. The new version is deployed to the idle one (green), QA tests it there, then traffic is switched; rollback is switching back. For QA it means testing in a production-like setup with zero downtime.
>
> **Example:** Run smoke and regression on green before the festival-sale release goes live.

44. What is the difference between CI/CD for backend APIs and for frontend UI testing?

> **Answer:** Backend/API pipelines run fast tests without a browser (API, contract, DB) on every commit. UI pipelines need browsers/Grid, are slower and flakier, so run smoke on commits and full regression nightly; frontend pipelines also add lint, build and cross-browser checks.
>
> **Example:** API regression on every commit; UI regression every night.

45. How do you test a CI/CD pipeline, its rollback mechanism and its scheduling? **[asked x3 in your bank]**

> **Answer:** Test the pipeline in a non-production branch/job with passing and failing builds: check triggers, stage order, failure handling and notifications. For rollback deploy the previous version (or switch blue-green) and run smoke tests; for scheduling confirm the cron time and time zone with a short interval first.
>
> **Example:** A deliberately failing test must stop the pipeline and send the Slack alert.

46. How do you schedule deployments?

> **Answer:** Use scheduled triggers (cron in Jenkins or schedule in GitHub Actions), release windows and manual approval gates, and run smoke tests right after the deployment.
>
> **Example:** Production deployment every Friday 10 PM after manager approval.

```
triggers { cron('0 22 * * 5') }
// stage('Approve') { steps { input message: 'Deploy to production?' } }
```

Jenkins: Jobs, Pipelines, Triggers & Scheduling (2)

47. What are shared libraries in Jenkins?

> **Answer:** A shared library is reusable pipeline code (Groovy) kept in a separate Git repository and loaded with @Library, so many projects reuse common steps.
>
> **Example:** A runTests(env) step used by all automation pipelines.

```
@Library('qa-shared-lib') _
pipeline {
  agent any
  stages { stage('Test') { steps { runTests(env: 'qa') } } }
}
```

48. How do you secure Jenkins for QA projects?

> **Answer:** Enable authentication and role-based access, give least privilege, keep secrets in the credentials store (not in the Jenkinsfile), use HTTPS, restrict script approvals, keep plugins updated and audit access.
>
> **Example:** Test credentials are injected from Jenkins credentials and masked in logs.

```
withCredentials([usernamePassword(credentialsId: 'qa-user',
    usernameVariable: 'USER', passwordVariable: 'PASS')]) {
  sh 'mvn test -Duser=$USER -Dpass=$PASS'
}
```

Docker for QA Automation (8)

49. What are common Docker commands every QA should know?

> **Answer:** Common commands: docker pull, images, run, ps, ps -a, stop, rm, rmi, logs, exec, build, cp and docker compose up/down.
>
> **Example:** docker logs to see why the Selenium container failed.

```
docker pull selenium/standalone-chrome
docker ps -a
docker logs <container>
docker exec -it <container> bash
docker compose up -d
```

50. What is a Docker Volume and how do you persist test results in containers? **[asked x2 in your bank]**

> **Answer:** A volume is Docker-managed storage that outlives a container. Mount a host folder or a named volume to keep test reports, screenshots and data after the container is removed.
>
> **Example:** Reports written in the container appear in ./target on the Jenkins agent.

```
docker run --rm -v $(pwd)/target:/app/target shop-tests
docker volume create test-results
```

51. How do you handle environment variables in Docker for QA tests?

> **Answer:** Pass variables with -e, --env-file or the compose environment section and read them in the tests; keep secrets out of the image and take them from CI credentials.
>
> **Example:** ENV=qa and BROWSER=chrome passed to the test container.

```
docker run --rm -e ENV=qa -e BROWSER=chrome shop-tests
docker run --rm --env-file qa.env shop-tests
```

52. How do you debug test failures in Docker containers?

> **Answer:** Read docker logs, open a shell with docker exec, keep the container (no --rm) to inspect it, mount a volume for screenshots, watch the browser through the VNC/noVNC port of the Selenium image and check memory (shm_size) and CPU.
>
> **Example:** Chrome crashing in a container is fixed by increasing --shm-size to 2g.

```
docker logs shop-tests
docker exec -it shop-tests bash
docker stats
```

53. How do you run database containers for test automation?

> **Answer:** Start the database image with environment variables and a mapped port, load seed data from an init script, or use Testcontainers from the Java tests.
>
> **Example:** A fresh MySQL container with shop data for every test run.

```
docker run -d --name testdb -e MYSQL_ROOT_PASSWORD=pass -e MYSQL_DATABASE=shop \
  -p 3306:3306 -v $(pwd)/init:/docker-entrypoint-initdb.d mysql:8
```

54. What is the difference between docker run and docker exec?

> **Answer:** docker run creates and starts a new container from an image. docker exec runs a command inside an already running container.
>
> **Example:** run starts the test container; exec opens a shell in it to debug.

```
docker run -d --name tests shop-tests sleep 3600
docker exec -it tests bash
```

55. How do you optimize Docker images for faster QA pipeline execution?

> **Answer:** Use small base images, multi-stage builds, layer caching (copy pom.xml and download dependencies before the source), a .dockerignore file, pinned versions and pre-built images from a registry.
>
> **Example:** Dependency layer is cached, so a code change rebuilds in seconds.

56. What is Docker Hub and how do QA engineers use it?

> **Answer:** Docker Hub is a public image registry. QA pulls official images (selenium, maven, mysql), pushes its own test-runner image to a repository and uses tags for versions.
>
> **Example:** Pull selenium/standalone-chrome and push shop-tests:1.0 for the CI agents.

```
docker login
docker pull selenium/standalone-chrome
docker tag shop-tests myorg/shop-tests:1.0
docker push myorg/shop-tests:1.0
```

Linux / CLI Basics for QA (2)

57. How do you see running processes and check disk and memory usage? **[asked x3 in your bank]**

> **Answer:** Use ps or top/htop for processes, df -h for disk, du -sh for folder size and free -m for memory.
>
> **Example:** Check the test server is not out of disk space before a long regression run.

```
ps -ef | grep java
top
df -h
du -sh /var/log/*
free -m
```

58. How do you find a file in Linux and check network connectivity? **[asked x2 in your bank]**

> **Answer:** Find files with find (or locate). For network use ping, curl -I for HTTP, ss/netstat for listening ports and nc -zv for a port check.
>
> **Example:** Check the QA app is reachable on port 8080.

```
find / -name "testng.xml" 2>/dev/null
ping -c 3 qa.shop.com
curl -I https://qa.shop.com/health
nc -zv qa.shop.com 8080
```

Section 4: Low Priority / Quick Revision (4 questions)

Basic commands and rarely asked details. Skim them so you are never caught off guard.

Jenkins: Jobs, Pipelines, Triggers & Scheduling (1)

59. What are the two components Jenkins is integrated with?

> **Answer:** The usual pairing is source control (Git/GitHub) and a build tool (Maven); Jenkins is also integrated with test frameworks and tools such as TestNG, Selenium Grid and Docker. The question is vague, so state what you used.
>
> **Example:** Jenkins pulls from Git and builds with Maven.

Docker for QA Automation (1)

60. What is the difference between docker ps and docker images? How do you remove unused containers and images? **[asked x2 in your bank]**

> **Answer:** docker ps lists running containers (-a for all); docker images lists local images. Remove with docker rm and docker rmi, or clean unused items with prune.
>
> **Example:** Free disk space on the CI agent by pruning old images.

```
docker ps -a
docker images
docker rm <container>
docker rmi <image>
docker system prune -f
```

Linux / CLI Basics for QA (2)

61. How do you list files, check and change the current directory, and create directories and files? **[asked x5 in your bank]**

> **Answer:** Use ls to list, pwd to show the current directory, cd to change it, mkdir to create directories and touch to create files.
>
> **Example:** Create a results folder before a test run.

```
ls -l
pwd
cd /opt/tests
mkdir reports
touch reports/run.log
```

62. How do you view file contents, and copy, move and delete files? **[asked x2 in your bank]**

> **Answer:** Use cat, less, head and tail to view files; cp to copy, mv to move or rename, rm to delete (rm -r for folders).
>
> **Example:** Copy the latest report to a shared folder and delete old logs.

```
cat results.txt
head -n 20 app.log
cp report.html /shared/
mv old.log archive/
rm temp.log
```